MATERIAL DE APOYO

PLANEAR LAS ACTIVIDADES DE CONSTRUCCIÓN DEL SOFTWARE

CONCEPTOS Y FUNDAMENTOS TEÓRICOS

¿Qué significa planear las actividades de construcción?

La planeación de la construcción consiste en definir, organizar y estimar el trabajo necesario para transformar los artefactos de diseño en componentes ejecutables del software.

El resultado no es solamente una lista de tareas. Un plan útil debe relacionar cada actividad con el diseño que la origina, el responsable de ejecutarla, los recursos requeridos, sus dependencias, el tiempo estimado y el producto que permitirá verificar su realización.

La planeación debe responder, como mínimo, las siguientes preguntas:

- ¿Qué se debe construir?
- ¿Qué actividades se deben realizar?
- ¿En qué orden deben ejecutarse?
- ¿Quién será responsable?
- ¿Qué recursos y herramientas se necesitan?
- ¿Cuánto esfuerzo y tiempo se estima?
- ¿Qué productos se entregarán?
- ¿Cómo se verificará el cumplimiento?

Relación entre diseño y construcción

El diseño describe cómo estará estructurada y cómo funcionará la solución. La construcción implementa esa estructura y ese comportamiento mediante código, configuraciones y otros artefactos técnicos.

Artefacto de diseño	Información que aporta	Actividades de construcción posibles
Diagrama de arquitectura	Capas, componentes y relaciones	Crear la estructura del proyecto y configurar los componentes
Diagrama de clases	Clases, atributos, operaciones y relaciones	Implementar clases, modelos y servicios

Modelo de datos	Entidades, atributos, relaciones y restricciones	Crear tablas, migraciones, modelos y validaciones
Diagrama de casos de uso	Funcionalidades y actores	Implementar funcionalidades y servicios asociados
Diseño de interfaces	Pantallas, navegación y componentes visuales	Construir formularios, vistas y componentes de interfaz
Especificación de API	Endpoints, parámetros, respuestas y reglas	Implementar rutas, controladores y validaciones
Requisitos no funcionales	Seguridad, rendimiento, disponibilidad y mantenibilidad	Configurar controles de acceso, registros, pruebas y optimizaciones

Descomposición del trabajo

La descomposición consiste en dividir el trabajo general en unidades más pequeñas que puedan estimarse, asignarse, ejecutarse y verificarse.

Una forma de organizarlo es mediante una estructura de desglose del trabajo (EDT o WBS, por sus siglas en inglés).

Ejemplo para el sistema de inventario:

Nivel 1. Construcción del sistema de inventario.

- 1.1. **Preparación del entorno**
  - Configurar el repositorio.
  - Instalar dependencias.
  - Crear la estructura del proyecto.
- 1.2. **Construcción de la base de datos**
  - Crear el esquema.
  - Definir las tablas y restricciones.
  - Crear los datos iniciales de prueba.
- 1.3. **Construcción del backend**
  - Implementar modelos.
  - Implementar servicios.
  - Crear endpoints.
  - Implementar validaciones.

- 1.4. **Construcción del frontend**
    - Crear componentes visuales.
    - Implementar formularios.
    - Integrar la API.
- 1.5**. Verificación**
    - Ejecutar pruebas unitarias.
    - Ejecutar pruebas de integración.
    - Corregir defectos.
- 1.6. **Integración y entrega**
    - Integrar componentes.
    - Preparar la versión candidata.
    - Documentar la entrega.

Cada actividad debe tener un alcance claro y un producto verificable

Historias de usuario, tareas técnicas y criterios de aceptación

En equipos que utilizan **Scrum u otro enfoque ágil**, las historias de usuario ayudan a expresar una necesidad desde la perspectiva del usuario. Para la construcción, normalmente se descomponen en tareas técnicas.

Ejemplo: Historia de usuario

Como administrador de una tienda, quiero registrar productos para mantener actualizado el inventario.

Criterios de aceptación

- El sistema permite registrar el código, nombre, precio y categoría.
- El código del producto no puede estar duplicado.
- El precio debe ser mayor que cero.
- El sistema confirma el registro cuando los datos son válidos.
- El sistema informa los errores de validación.

Tareas técnicas derivadas

1. Crear o ajustar la tabla de productos.
2. Implementar el modelo de producto.
3. Crear el servicio de registro.
4. Implementar el endpoint correspondiente.
5. Construir el formulario de registro.

6. Integrar el formulario con el backend.
7. Elaborar y ejecutar pruebas.
8. Documentar la funcionalidad.

Los criterios de aceptación permiten determinar cuándo la funcionalidad satisface la necesidad descrita.

Estimación del esfuerzo y la duración

Es importante distinguir los siguientes conceptos:

Esfuerzo: cantidad de trabajo requerida para realizar una actividad. Puede expresarse en horas-persona o días-persona.

Duración: tiempo calendario transcurrido desde el inicio hasta la finalización de una actividad. Por ejemplo, una tarea puede requerir 8 horas-persona de esfuerzo. Si una persona dispone de 4 horas diarias para trabajar en ella, la duración aproximada será de 2 días laborables, siempre que no existan interrupciones ni dependencias adicionales.

La duración no siempre disminuye al asignar más personas. Algunas tareas necesitan coordinación, conocimientos especializados o una secuencia de ejecución que limita el trabajo paralelo.

Técnicas de estimación

Técnica	Descripción	¿Cuándo utilizarla?
Juicio de expertos	Estimar con base en experiencia técnica	Cuando hay personal con experiencia en tareas similares
Estimación análoga	Comparar con actividades de proyectos anteriores	Cuando existen registros históricos
Estimación paramétrica	Aplicar una relación cuantitativa, como tiempo por unidad	Cuando se dispone de datos confiables
Estimación de tres puntos	Considerar escenarios optimista, más probable y pesimista	Cuando existe incertidumbre
Planning Poker	Estimación relativa y colaborativa	Cuando el equipo utiliza métodos ágiles

El resultado es una estimación, no una garantía. Se debe ajustar con base en el contexto, la experiencia del equipo y las incertidumbres identificadas.

Dependencias y secuenciación de actividades

Las dependencias indican qué actividades deben completarse o avanzar antes de que otras puedan ejecutarse plenamente.

Ejemplo:

- La estructura de la base de datos debe estar definida antes de implementar determinadas operaciones de persistencia.
- La especificación del endpoint debe conocerse antes de completar su integración con el frontend.
- Las pruebas de integración requieren que los componentes involucrados estén disponibles.
- La entrega de una versión integrada depende de que los componentes esenciales estén integrados y verificados.

Las dependencias no siempre obligan a detener todo el trabajo. Algunas tareas pueden comenzar en paralelo si existen contratos de interfaz claros, datos simulados o ambientes independientes.

Tipos básicos de relación

- **Fin a inicio:** una actividad debe terminar para que otra comience.
- **Inicio a inicio:** dos actividades pueden comenzar cuando se inicia una actividad de referencia.
- **Fin a fin:** dos actividades deben terminar en una relación acordada.
- **Inicio a fin:** una actividad no puede terminar hasta que otra haya comenzado; es una relación menos frecuente.

La secuenciación permite construir un cronograma realista y reconocer actividades que pueden ejecutarse en paralelo.

Asignación de responsables y recursos

La planeación debe establecer quién ejecutará, revisará o aprobará cada actividad.

Los recursos pueden incluir:

- Personas y roles técnicos.
- Computadores y ambientes de desarrollo.
- Repositorios de código.
- Sistemas gestores de bases de datos.
- Herramientas de modelado y gestión.
- Servicios de despliegue y pruebas.
- Documentación y datos de prueba.
- Herramientas de inteligencia artificial autorizadas.

En equipos pequeños, una persona puede desempeñar varios roles. Sin embargo, se deben definir claramente las responsabilidades para evitar que actividades importantes queden sin atender.

Una matriz **RACI** puede apoyar esta organización:

- R (Responsible): ejecuta el trabajo.
- A (Accountable): responde por el resultado final y su aprobación.
- C (Consulted): aporta información o asesoría.

- I (Informed): recibe información sobre el avance.

Cada actividad debe tener una persona o rol responsable de ejecutarla y una responsabilidad de aprobación claramente identificada.

Metodologías y herramientas para organizar el trabajo

Enfoques de trabajo

Enfoque secuencial: organiza las actividades en etapas con entregables definidos. Puede resultar útil cuando el alcance y los criterios de aceptación están suficientemente establecidos.

Enfoque ágil: organiza el trabajo en incrementos pequeños, con priorización, seguimiento frecuente y adaptación ante cambios.

Enfoque híbrido: combina una planificación general con ciclos iterativos de construcción y revisión.

La selección depende del tamaño del proyecto, los requisitos, la incertidumbre, el equipo y las condiciones de entrega.

Herramientas de apoyo

Herramienta	Uso en la planeación
Trello	Organizar tareas mediante tarjetas y columnas de seguimiento
Jira	Gestionar backlog, historias, tareas, sprints e incidencias
GitHub Projects	Organizar trabajo relacionado con repositorios y desarrollo
Microsoft Project u otras herramientas de cronograma	Definir actividades, duraciones, dependencias e hitos
Git y GitHub/GitLab	Gestionar versiones, ramas, cambios e integración del código
diagrams.net	Consultar o elaborar diagramas de apoyo
Hojas de cálculo	Registrar estimaciones, responsables y cronogramas

La herramienta no reemplaza la planeación. Una lista de tareas en un tablero solo constituye un plan útil cuando las tareas están bien definidas, estimadas, relacionadas con el diseño y sujetas a seguimiento.

Hitos, entregables y definición de terminado

Un **entregable** es un producto verificable generado durante el trabajo. Puede ser un módulo, un endpoint, un modelo de datos, una interfaz o un informe de pruebas.

Un **hito** es un punto significativo de control que representa un avance importante del proyecto. Por ejemplo, la finalización de la estructura del backend o la integración de una funcionalidad completa.

La **definición de terminado** establece las condiciones mínimas para considerar que una tarea está completa.

Ejemplo de definición de terminado para un endpoint:

- Cumple el contrato de API establecido.
- Incluye las validaciones necesarias.
- Maneja los errores esperados.
- Cuenta con pruebas pertinentes ejecutadas.
- El código fue integrado al repositorio.
- La revisión técnica requerida fue realizada.
- La documentación está actualizada cuando aplica.

Estas condiciones deben adaptarse al proyecto. No basta con que el código funcione en el computador del desarrollador.

Riesgos en la planeación de la construcción

Un riesgo es una situación incierta que puede afectar los objetivos del proyecto.

Riesgo	Posible consecuencia	Acción preventiva
Estimaciones demasiado optimistas	Retrasos en las entregas	Utilizar estimaciones justificadas y revisar supuestos
Cambios en el diseño	Retrabajo	Controlar cambios y analizar su impacto
Dependencias no identificadas	Bloqueos	Definir interfaces y secuenciar tareas
Falta de conocimientos técnicos	Baja productividad	Capacitación, tutoría y pruebas de concepto
Integración tardía	Defectos difíciles de detectar	Integrar y verificar progresivamente
Uso de código generado por IA sin revisión	Errores o vulnerabilidades	Revisar, probar y verificar licencias y seguridad

El equipo debe identificar los riesgos más importantes, estimar su probabilidad e impacto y proponer acciones para reducirlos.

Uso de inteligencia artificial en la planeación

Las herramientas de IA pueden apoyar la descomposición de historias de usuario, sugerir tareas técnicas, proponer estimaciones iniciales, identificar dependencias y generar borradores de criterios de terminación.

Ejemplo de prompt:

Actúa como líder técnico de un equipo de desarrollo de software. A partir de la siguiente historia de usuario y sus criterios de aceptación, propón una lista de tareas técnicas para construir la funcionalidad. Para cada tarea, indica el componente involucrado, la dependencia, el entregable verificable, los conocimientos técnicos necesarios y una estimación inicial de esfuerzo en horas-persona. Explica los supuestos y señala las incertidumbres. No invente requisitos que no estén en la descripción; identifica explícitamente la información faltante.

El resultado de la IA debe revisarse con criterio técnico. Las estimaciones generadas automáticamente no sustituyen la experiencia del equipo ni los datos históricos.

Buenas prácticas:

- No introducir contraseñas, claves, datos personales ni información confidencial en servicios no autorizados.
- Verificar la correspondencia entre las sugerencias y los diseños establecidos.
- Revisar el código o las soluciones técnicas propuestas antes de incorporarlas al proyecto.
- Contrastar las estimaciones con la experiencia del equipo.
- Documentar las decisiones relevantes y los supuestos.
- Respetar las políticas institucionales y las condiciones de uso de las herramientas.

EJEMPLO PRÁCTICO DESARROLLADO

Caso: módulo de gestión de productos

El equipo debe construir una funcionalidad que permita registrar productos en una tienda virtual.

Información de entrada

Requerimiento funcional: el administrador debe poder registrar un producto con código único, nombre, precio y categoría.

Diseño establecido:

- Entidad Producto con atributos definidos.
- Código de producto único.
- Interfaz web para registrar productos.
- Endpoint para crear productos.
- Validación de campos obligatorios.
- Persistencia en una base de datos relacional.

Tecnologías propuestas para el ejercicio: React, Node.js, una API REST y MySQL. Pueden sustituirse por las tecnologías definidas en el proyecto real.

Descomposición y estimación

Las cifras siguientes son ilustrativas y se expresan en horas-persona. El instructor debe ajustarlas según el nivel de los aprendices y las condiciones del proyecto.

ID	Actividad	Dependencia	Esfuerzo estimado	Entregable
A1	Revisar el diseño y confirmar el contrato de la API	Ninguna	2 h	Especificación revisada
A2	Preparar el repositorio y la estructura del proyecto	Ninguna	2 h	Proyecto configurado
A3	Crear la tabla y restricciones de productos	A1	3 h	Script o migración
A4	Implementar modelo y servicio de productos	A3	4 h	Modelo y servicio
A5	Implementar endpoint y validaciones	A1, A4	4 h	API funcional
A6	Construir el formulario de registro	A1, A2	4 h	Interfaz implementada
A7	Integrar el formulario con la API	A5, A6	3 h	Funcionalidad integrada
A8	Elaborar y ejecutar pruebas	A5, A7	4 h	Evidencias de pruebas
A9	Corregir defectos y preparar la entrega	A8	3 h	Versión verificada

Esfuerzo total estimado: 29 horas-persona.

Este total no representa necesariamente 29 horas consecutivas de calendario. La duración real depende de la disponibilidad de los integrantes, las dependencias y las actividades que puedan ejecutarse en paralelo.

Asignación de responsables

Actividad	Responsable principal	Apoyo
A1	Analista o líder técnico	Equipo de desarrollo
A2	Desarrollador responsable del entorno	Líder técnico
A3	Desarrollador backend	Responsable de datos
A4	Desarrollador backend	Líder técnico
A5	Desarrollador backend	Responsable de pruebas
A6	Desarrollador frontend	Diseñador de interfaces
A7	Desarrolladores frontend y backend	Equipo
A8	Responsable de pruebas	Desarrolladores
A9	Equipo de desarrollo	Líder técnico

Los roles pueden ser asumidos por aprendices distintos o compartidos, de acuerdo con el tamaño del equipo.

Criterios para verificar la planeación

El plan se considera técnicamente coherente cuando:

- Las tareas se relacionan con los diseños y requerimientos.
- Los entregables son verificables.
- Las dependencias están identificadas.
- Las estimaciones tienen una justificación.
- Las responsabilidades están definidas.
- Se han incluido las actividades de integración y pruebas.
- Los riesgos relevantes cuentan con acciones de tratamiento.
- Las tareas tienen condiciones claras de terminación.

Referencias

- Schwaber, K. y Sutherland, J. *The Scrum Guide*. Guía de referencia para comprender los elementos de Scrum, el Sprint Backlog y la planificación del trabajo. Disponible en: https://scrumguides.org/
- Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. Referente para la gestión, planificación y seguimiento de proyectos.
- Atlassian. Recursos de Scrum, gestión de proyectos y seguimiento de tareas. Disponible en: https://www.atlassian.com/agile
- GitHub Docs. Documentación sobre proyectos, repositorios y colaboración en el desarrollo. Disponible en: https://docs.github.com/
- OWASP Foundation. Recursos de seguridad de aplicaciones web. Disponible en: https://owasp.org/
- Servicio Nacional de Aprendizaje (SENA). Diseño curricular vigente del programa Tecnólogo en Análisis y Desarrollo de Software, proyecto formativo y documentos institucionales de planeación de la formación.

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CTPI-CAUCA	20-10-2026